

# myMove Agile Sprint Roadmap - Prototype Version

## 20-Week Development Plan for Physical Prototype Model

---

### Project Overview

**System Type:** Physical prototype demonstration model with working hardware-software integration

**Hardware:** 1 ESP32 + 4 ultrasonic sensors in cardboard/foam board parking model

**Software:** Flutter mobile app + Firebase backend

**Target:** Demonstrate proof-of-concept for smart parking system

---

### Sprint 0: Foundation & Procurement (Week 1-2)

#### Goals:

- Development environment ready
- Hardware components ordered and received
- Project infrastructure set up

#### Tasks:

##### Week 1: Software Setup

- ☐ Install Flutter SDK (latest stable)
- ☐ Install Android Studio with Flutter plugin
- ☐ Install Arduino IDE
- ☐ Add ESP32 board support to Arduino IDE
- ☐ Install required Arduino libraries (Firebase ESP Client)
- ☐ Create Firebase project "myMove"
- ☐ Set up Git repository
- ☐ Create Flutter project structure

##### Week 2: Hardware Procurement & Initial Testing

- ☒ Order ESP32 (1 unit) - ✓ Ordered RM 32.64
- ☒ Order HC-SR04 sensors (4 units) - ✓ Ordered RM 37 (RM 9.25 × 4)
- ☒ Order breadboard - ✓ Ordered RM 2.82
- ☒ Order jumper wires - ✓ Ordered RM 10.20 (RM 5.10 × 2)
- ☒ Order LEDs (Green/Red) - ✓ Ordered RM 2.40 (10pcs)
- ☐ Get foam board/cardboard for prototype box (RM 15-20)

- ☐ Buy 4 toy cars (RM 20-30)
- ☐ Get paint/markers for parking layout (RM 15)
- ☐ Test ESP32 with 1 sensor on breadboard
- ☐ Verify WiFi connection works

**Deliverables:**

- ☒ All development tools installed
- ☒ Firebase project created
- ☒ Hardware components received
- ☒ Single sensor tested and working
- ☒ Git repository initialized

**Success Criteria:**

- ESP32 connects to WiFi successfully
  - One sensor measures distance accurately
  - Data appears in Firebase Console
- 

**Sprint 1: Basic Hardware Integration (Week 3-4)****Goal:**

Build and test complete 4-sensor prototype model with ESP32

**User Story:**

"As a system tester, I want to see real-time status of 4 parking spots in Firebase when I place/remove toy cars"

**Tasks:****Week 3: Build Physical Prototype**

- ☐ Design parking layout (4 spots: A01-A04)
- ☐ Cut and assemble foam board/cardboard box (40cm × 30cm)
- ☐ Paint parking layout (gray base, white lines)
- ☐ Mark spot numbers (A01, A02, A03, A04)
- ☐ Add building sign "Prototype Demo - Floor 1"
- ☐ Create mounting positions for 4 sensors
- ☐ Mount sensors 10-15cm above each parking spot
- ☐ Secure mounting with hot glue/tape

**Week 4: Wire and Program ESP32**

- ☐ Wire all 4 sensors to ESP32 using breadboard
  - Sensor 1 → GPIO 5, 18
  - Sensor 2 → GPIO 19, 21
  - Sensor 3 → GPIO 22, 23
  - Sensor 4 → GPIO 25, 26
- ☐ Upload 4-sensor code to ESP32
- ☐ Configure WiFi credentials
- ☐ Set up Firebase Realtime Database structure
- ☐ Test each sensor individually
- ☐ Calibrate detection thresholds for toy cars (15cm)
- ☐ Run 24-hour stability test

#### **Deliverables:**

- ☒ Complete physical prototype model
- ☒ All 4 sensors detecting accurately
- ☒ Real-time data streaming to Firebase
- ☒ Can demonstrate occupied/available status

#### **Success Criteria:**

- Place toy car → Firebase shows "occupied: true" within 2 seconds
- Remove car → Firebase shows "occupied: false" within 2 seconds
- 95%+ detection accuracy
- System runs continuously for 24 hours without crashes

#### **Demo to Supervisor:**

Show working prototype with Serial Monitor and Firebase Console

---

## **Sprint 2: Flutter Foundation (Week 5-6)**

#### **Goal:**

Create Flutter app skeleton and display real-time parking data

#### **User Story:**

"As a driver, I want to see the 4 parking spots from the prototype and their real-time availability status"

## Tasks:

### Week 5: App Structure & Firebase Connection

- ☐ Create Flutter project "my\_move"
- ☐ Set up folder structure (models, services, screens, widgets)
- ☐ Add Firebase packages to pubspec.yaml
- ☐ Configure Firebase for Android
- ☐ Create data models (ParkingSpot, Building)
- ☐ Create ParkingService to stream Firebase data
- ☐ Test Firebase connection in simple test screen

### Week 6: Basic Parking Display UI

- ☐ Design home screen UI (Figma → Flutter)
- ☐ Create parking grid widget (4 spots in 2×2 layout)
- ☐ Implement color coding (green=available, red=occupied)
- ☐ Stream parking data from Firebase Realtime Database
- ☐ Update UI when sensor data changes
- ☐ Add spot labels (A01, A02, A03, A04)
- ☐ Test with physical prototype

## Deliverables:

- ☒ Flutter app displays 4 parking spots
- ☒ Real-time updates when toy cars placed/removed
- ☒ Color-coded status indicators
- ☒ Smooth UI performance

## Success Criteria:

- App connects to Firebase on first launch
- Parking status updates within 2 seconds
- No UI lag or freezing
- Works on Android device

## Demo to Supervisor:

Show app updating as you place/remove toy cars on prototype

---

## **Sprint 3: User Authentication (Week 7-8)**

### **Goal:**

Implement user registration and login system

### **User Story:**

"As a driver, I want to create an account and log in so I can make parking reservations"

### **Tasks:**

#### **Week 7: Authentication UI**

- ☐ Design onboarding screen
- ☐ Create login screen UI
- ☐ Create registration screen UI
- ☐ Add form validation (email, password)
- ☐ Create input widgets (email field, password field)
- ☐ Add password visibility toggle
- ☐ Design error messages

#### **Week 8: Firebase Authentication**

- ☐ Set up Firebase Authentication
- ☐ Create AuthProvider with state management
- ☐ Implement email/password registration
- ☐ Save user data to Firestore (name, email, vehicle number)
- ☐ Implement login functionality
- ☐ Add logout feature
- ☐ Handle authentication errors
- ☐ Implement session persistence (stay logged in)
- ☐ Create user profile screen

### **Deliverables:**

- ☒ Users can register new accounts
- ☒ Users can log in
- ☒ User data saved in Firestore
- ☒ Session persists across app restarts

### **Success Criteria:**

- Registration flow works smoothly
- Login takes <2 seconds

- Proper error messages shown
- User data stored correctly

### **Demo to Supervisor:**

Register → Login → View profile

---

## **Sprint 4: Building & Spot Selection (Week 9-10)**

### **Goal:**

Users can select building and view detailed parking layout

### **User Story:**

"As a driver, I want to select 'Prototype Demo' building and see which spots are available"

### **Tasks:**

#### **Week 9: Building Information**

- ☐ Create Building data model
- ☐ Add "Prototype Demo" building to Firestore
- ☐ Create building details screen
- ☐ Display building info (name, address, rates, hours)
- ☐ Show total spots and available spots count
- ☐ Add "View Parking" button
- ☐ Implement navigation to parking layout

#### **Week 10: Interactive Parking Layout**

- ☐ Create parking spot selection screen
- ☐ Display 4 spots in grid (2×2 or 1×4 layout)
- ☐ Add floor selector (even if only 1 floor)
- ☐ Make spots clickable
- ☐ Show spot details on tap
- ☐ Highlight selected spot
- ☐ Disable occupied/reserved spots
- ☐ Add spot legends (Available/Occupied/Reserved)

### **Deliverables:**

- ☒ Building details screen functional
- ☒ Interactive parking grid showing 4 spots

- ☒ Real-time status reflected in UI
- ☒ Users can select available spots

#### **Success Criteria:**

- Grid updates within 2 seconds when prototype changes
- Occupied spots clearly disabled
- Selected spot highlighted
- Smooth navigation flow

#### **Demo to Supervisor:**

Navigate: Building list → Building details → Parking layout (live updates)

---

### **Sprint 5: Booking & Reservation (Week 11-12)**

#### **Goal:**

Users can reserve parking spots with date/time selection

#### **User Story:**

"As a driver, I want to reserve spot A02 for tomorrow 2PM-4PM"

#### **Tasks:**

##### **Week 11: Booking Form**

- ☐ Create booking screen UI
- ☐ Add date picker widget
- ☐ Add time picker (start/end time)
- ☐ Implement duration calculator
- ☐ Calculate parking fees
- ☐ Show vehicle selection (from user profile)
- ☐ Create Booking data model
- ☐ Add BookingProvider state management

##### **Week 12: Reservation Logic**

- ☐ Create BookingService
- ☐ Implement spot reservation in Firebase
- ☐ Update Realtime DB (set reserved: true)
- ☐ Create booking record in Firestore
- ☐ Add reservation validation (spot must be available)

- ☐ Implement 10-minute auto-release for unpaid reservations
- ☐ Show booking confirmation screen
- ☐ Add booking to user's history

#### **Deliverables:**

- ☒ Complete booking flow functional
- ☒ Reserved spots shown correctly in app
- ☒ Other users cannot select reserved spots
- ☒ Booking data saved to Firestore

#### **Success Criteria:**

- User can select date/time and reserve spot
- Reservation reflects in Firebase within 1 second
- Spot shows as "Reserved" (orange) in UI
- Booking appears in user's booking list

#### **Demo to Supervisor:**

Select spot → Choose date/time → Reserve → See confirmation

---

## **Sprint 6: Payment Integration (Week 13-14)**

#### **Goal:**

Process parking payments through Stripe

#### **User Story:**

"As a driver, I want to pay for my booking using credit card"

#### **Tasks:**

#### **Week 13: Stripe Setup**

- ☐ Create Stripe test account
- ☐ Get API keys (test mode)
- ☐ Install flutter\_stripe package
- ☐ Configure Stripe in Flutter app
- ☐ Create Cloud Function: createPaymentIntent
- ☐ Test payment intent creation

#### **Week 14: Payment UI & Flow**



- ☐ Create payment method screen
- ☐ Implement card input form
- ☐ Show payment summary (breakdown, total)
- ☐ Create review screen before payment
- ☐ Implement payment confirmation
- ☐ Handle payment success
- ☐ Handle payment failure
- ☐ Update booking status after payment
- ☐ Generate payment receipt
- ☐ Send confirmation notification

#### **Deliverables:**

- ☒ Working payment flow (test mode)
- ☒ Card details captured securely
- ☒ Payment processed through Stripe
- ☒ Booking activated after successful payment

#### **Success Criteria:**

- Test card payment succeeds
- Booking status updates to "active"
- Receipt generated with transaction ID
- Error handling for failed payments

#### **Demo to Supervisor:**

Complete booking → Enter test card → Pay → Confirmation

---

### **Sprint 7: Active Booking & Timer (Week 15-16)**

#### **Goal:**

Display active parking session with countdown timer

#### **User Story:**

"As a driver, I want to see how much parking time I have left and extend if needed"

#### **Tasks:**

#### **Week 15: Active Booking Screen**

- ☐ Create active booking/parking timer screen
- ☐ Display circular countdown timer
- ☐ Show remaining time (hours:minutes)
- ☐ Display parking spot details
- ☐ Show parking duration
- ☐ Add "End Parking" button
- ☐ Implement early checkout
- ☐ Release spot when parking ends

### **Week 16: Time Extension**

- ☐ Create extend parking screen
- ☐ Add time slider (add 30min, 1hr, 2hr)
- ☐ Recalculate fees for extension
- ☐ Process extension payment
- ☐ Update booking end time
- ☐ Update Firebase with new end time
- ☐ Implement parking expiry notifications
- ☐ Create Cloud Function for 15-min reminder

### **Deliverables:**

- ☒ Countdown timer showing remaining time
- ☒ Can extend parking time
- ☒ Additional payment processed
- ☒ Notifications sent before expiry

### **Success Criteria:**

- Timer accurate within 1 second
- Extension updates immediately
- Notifications sent 15 minutes before expiry
- Spot released when parking ends

### **Demo to Supervisor:**

Show active booking → Extend time → Receive notification

---

## **Sprint 8: QR Code System (Week 17-18)**

### **Goal:**

Implement QR-based double parking solution

### **User Story:**

"As a blocked driver, I want to scan a QR code on the blocking car to contact the owner without sharing phone numbers"

### **Tasks:**

#### **Week 17: QR Generation**

- ☐ Install qr\_flutter package
- ☐ Generate unique QR code for each booking
- ☐ QR data format: "bookingId|vehicleNumber|userId"
- ☐ Create QR display screen
- ☐ Add "Show My QR" button to active booking
- ☐ Design QR display with spot info
- ☐ Print physical QR codes for toy cars (demo props)

#### **Week 18: QR Scanning & Contact**

- ☐ Install mobile\_scanner package
- ☐ Create QR scanner screen
- ☐ Add camera permission handling
- ☐ Implement QR code detection
- ☐ Parse scanned QR data
- ☐ Validate QR code authenticity
- ☐ Create contact options dialog (Message/Call)
- ☐ Link to communication features
- ☐ Add "Scan QR" button to home screen

### **Deliverables:**

- ☒ Each booking generates unique QR code
- ☒ Physical QR codes attached to toy cars
- ☒ App can scan QR codes
- ☒ Contact dialog appears with options

### **Success Criteria:**

- QR generation takes <1 second

- Camera opens smoothly
- QR scanning works within 2 seconds
- Correct user info extracted from QR

### **Demo to Supervisor:**

Show QR on phone → Scan with another phone → Contact options appear

---

## **Sprint 9: In-App Communication (Week 19-20)**

### **Goal:**

Enable messaging between drivers

### **User Story:**

"As a blocked driver, I want to message the car owner to ask them to move"

### **Tasks:**

#### **Week 19: Chat System**

- ☐ Design chat screen UI
- ☐ Create Message data model
- ☐ Set up Firestore for messages
- ☐ Implement send message functionality
- ☐ Implement receive messages (real-time)
- ☐ Display conversation history
- ☐ Add timestamp to messages
- ☐ Show read/unread status
- ☐ Add message notifications

#### **Week 20: Simplified Calling**

- ☐ Add "Call" button in chat
- ☐ Implement click-to-call (opens phone dialer)
- ☐ Use masked number system OR
- ☐ Add VoIP calling using Agora (if time permits)
- ☐ Test call notifications
- ☐ Add call history (optional)

### **Deliverables:**

- ☒ Working in-app chat
- ☒ Real-time message delivery

- ☒ Notifications for new messages
- ☒ Call functionality (basic or VoIP)

### **Success Criteria:**

- Messages delivered within 1 second
- Notifications work when app in background
- Chat history persists
- Can initiate calls from chat

### **Demo to Supervisor:**

Scan QR → Send message → Receive reply → Make call

---

## **Sprint 10: Additional Features (Week 21-22)**

### **Goal:**

Polish app with supporting features

### **Tasks:**

#### **Week 21: Navigation & Search**

- ☐ Add Google Maps to home screen
- ☐ Show prototype location on map
- ☐ Implement search functionality
- ☐ Add "Navigate" button (opens Google Maps)
- ☐ Create search results list
- ☐ Add filter options (distance, price)
- ☐ Implement booking history screen
- ☐ Show past bookings

#### **Week 22: LED Indicators (Hardware)**

- ☐ Wire 4 green LEDs to ESP32 (one per spot)
- ☐ Wire 4 red LEDs to ESP32 (one per spot)
- ☐ Update ESP32 code to control LEDs
- ☐ Green LED ON when spot available
- ☐ Red LED ON when spot occupied
- ☐ Mount LEDs on prototype model
- ☐ Test LED sync with app

## **Deliverables:**

- ☒ Search and navigation functional
- ☒ LED indicators working on prototype
- ☒ Visual confirmation of sensor status
- ☒ Booking history accessible

## **Success Criteria:**

- Search returns relevant results
  - Maps integration works smoothly
  - LEDs match app status exactly
  - Booking history shows all past bookings
- 

## **Sprint 11: Testing & Bug Fixes (Week 23-24)**

### **Goal:**

Comprehensive testing and issue resolution

### **Tasks:**

#### **Week 23: Functional Testing**

- ☐ Test all user flows end-to-end
- ☐ Test with poor WiFi connection
- ☐ Test Firebase offline persistence
- ☐ Test payment failures
- ☐ Test with multiple users simultaneously
- ☐ Check all error messages
- ☐ Verify data validation
- ☐ Test edge cases (expired bookings, etc.)

#### **Week 24: User Acceptance Testing**

- ☐ Recruit 10 test users (friends, classmates)
- ☐ Provide test scenarios
- ☐ Observe users interacting with prototype
- ☐ Collect feedback via Google Form
- ☐ Identify usability issues
- ☐ Fix critical bugs
- ☐ Improve UI/UX based on feedback

- ☐ Re-test after fixes

### **Deliverables:**

- ☒ All major bugs fixed
- ☒ User feedback documented
- ☒ System stability verified
- ☒ Test report with metrics

### **Success Criteria:**

- 80%+ user satisfaction rate
  - <5 critical bugs remaining
  - System uptime >95% during testing
  - Positive feedback on QR feature
- 

## **Sprint 12: Prototype Polish & Documentation (Week 25-26)**

### **Goal:**

Professional prototype finish and prepare for demonstration

### **Tasks:**

#### **Week 25: Prototype Enhancement**

- ☐ Clean up wiring (cable management)
- ☐ Add professional labels and signage
- ☐ Paint touch-ups if needed
- ☐ Add directional arrows on parking spots
- ☐ Create protective cover (optional clear acrylic)
- ☐ Build portable carrying case
- ☐ Add "myMove" branding to prototype
- ☐ Create demonstration cards/props

#### **Week 26: Demo Preparation**

- ☐ Write 5-minute demo script
- ☐ Practice demo presentation (10+ times)
- ☐ Prepare backup demo video
- ☐ Test setup time (<5 minutes)
- ☐ Prepare Firebase Console view

- ☐ Create demo checklist
- ☐ Test in presentation environment
- ☐ Prepare for Q&A (anticipated questions)

**Deliverables:**

- ☒ Professional-looking prototype
- ☒ Polished demonstration
- ☒ Backup materials ready
- ☒ Confident presentation

**Success Criteria:**

- Setup takes <5 minutes
  - Demo runs smoothly every time
  - Prototype looks professional
  - Can answer technical questions
- 

**Sprint 13-14: Thesis Writing (Week 27-30)****Goal:**

Complete FYP2 thesis documentation

**Week 27: Results Chapter**

- ☐ Document prototype specifications
- ☐ Include wiring diagrams
- ☐ Add sensor calibration data
- ☐ Present user testing results
- ☐ Create comparison tables
- ☐ Include screenshots of app
- ☐ Document Firebase data structure

**Week 28: Implementation Chapter**

- ☐ Explain hardware design decisions
- ☐ Document code architecture
- ☐ Include key code snippets
- ☐ Explain Firebase integration
- ☐ Describe Flutter implementation



- ☐ Add flowcharts and diagrams

## **Week 29: Testing & Evaluation Chapter**

- ☐ Present testing methodology
- ☐ Show test results (accuracy, performance)
- ☐ Include user feedback analysis
- ☐ Document bugs found and fixed
- ☐ Compare with objectives
- ☐ Evaluate system against requirements

## **Week 30: Conclusion & Proofreading**

- ☐ Summarize achievements
- ☐ Discuss limitations
- ☐ Propose future enhancements (camera upgrade, real deployment)
- ☐ Write acknowledgements
- ☐ Proofread entire thesis
- ☐ Format according to university guidelines
- ☐ Generate table of contents
- ☐ Submit draft to supervisor

### **Deliverables:**

- ☒ Complete thesis document
  - ☒ All chapters written
  - ☒ Figures and tables included
  - ☒ References properly cited
- 

## **Sprint 15: Final Preparation (Week 31-32)**

### **Week 31: Presentation Materials**

- ☐ Create PowerPoint slides (15-20 slides)
- ☐ Prepare demo video (backup)
- ☐ Design project poster (if required)
- ☐ Practice presentation (aim for 15-20 minutes)
- ☐ Prepare answers to common questions
- ☐ Test prototype one final time

## Week 32: Final Submission

- ☐ Final thesis proofreading
  - ☐ Print and bind thesis
  - ☐ Submit thesis
  - ☐ Prepare for viva/presentation
  - ☐ Final demo rehearsal
  - ☐ **GRADUATION!** 🎓
- 

## Agile Ceremonies for Solo Developer

### Weekly (Every Monday):

#### Sprint Planning (30 minutes)

- Review last week's progress
- Plan this week's tasks
- Identify potential blockers
- Set goals for the week

### Daily (5 minutes):

#### Personal Stand-up

- What did I accomplish yesterday?
- What will I do today?
- What's blocking me?
- Write in development log

### Every 2 Weeks (Friday):

#### Sprint Review (1 hour)

- Demo working features to supervisor/friend
- Show prototype progress
- Get feedback
- Record demo video

#### Sprint Retrospective (30 minutes)

- What went well?
- What could be improved?

- What will I change next sprint?
  - Update development log
- 

## **Risk Management & Mitigation**

### **Risk 1: ESP32 Hardware Failure**

**Probability:** Low

**Impact:** High

**Mitigation:**

- Order backup ESP32 (RM 30)
- Test thoroughly in Sprint 1
- Have manual Firebase update script as backup

### **Risk 2: WiFi Connectivity Issues During Demo**

**Probability:** Medium

**Impact:** High

**Mitigation:**

- Use phone hotspot as backup WiFi
- Pre-record demo video
- Have Firebase Console screenshots

### **Risk 3: Sensor Interference**

**Probability:** Low

**Impact:** Medium

**Mitigation:**

- Space sensors >10cm apart
- Add delay between readings (100ms)
- Test in demo environment beforehand

### **Risk 4: Time Constraints**

**Probability:** Medium

**Impact:** Medium

**Mitigation:**

- Prioritize core features (parking display, booking, QR)

- Mark optional features (VoIP calling, advanced navigation)
- Build MVP first, enhance later

### **Risk 5: User Testing Recruitment**

**Probability:** Low

**Impact:** Low

**Mitigation:**

- Test with classmates, family, friends
  - Offer small incentive (RM 5 coffee voucher)
  - Only need 10 users minimum
- 

### **Success Metrics**

**Technical Metrics:**

- Sensor detection accuracy: >95%
- App-Firebase sync time: <2 seconds
- System uptime: >95%
- Payment success rate: >98%

**User Metrics:**

- User satisfaction: >80%
- Task completion rate: >90%
- Would recommend: >75%

**Academic Metrics:**

- All objectives met: 100%
  - Thesis word count: 12,000-15,000 words
  - Supervisor approval: Required
  - Target grade: A- or better
-

## Deliverables Summary

### Hardware:

- ✓ Physical prototype model (40×30cm)
- ✓ 1 ESP32 + 4 sensors working
- ✓ LED indicators functional
- ✓ Portable and demo-ready

### Software:

- ✓ Flutter Android app
- ✓ Firebase backend (Auth, Firestore, Realtime DB)
- ✓ Stripe payment integration
- ✓ QR code system
- ✓ In-app messaging
- ✓ Real-time parking display

### Documentation:

- ✓ Complete thesis (60-70 pages)
  - ✓ Presentation slides
  - ✓ Demo video
  - ✓ User testing report
  - ✓ Source code on GitHub
- 

## What Makes This Agile?

### Iterative Development:

- Each sprint builds on previous
- Working features every 2 weeks
- Continuous testing and refinement

### Flexibility:

- Can adjust priorities based on progress
- Can drop optional features if time tight
- Can incorporate feedback immediately

### Risk Mitigation:

- Hardware tested in Sprint 1 (not Week 15!)

- User feedback in Sprint 11 (not at the end)
- Always have something working to demo

### **Continuous Delivery:**

- Sprint 2: Can show real-time parking
  - Sprint 5: Can show complete booking
  - Sprint 8: Can show full QR system
  - Sprint 10: Complete system demo
- 

## **Resource Allocation**

### **Time Distribution:**

- Hardware (Sprints 1, 10): 15%
- Core app development (Sprints 2-7): 40%
- Features & polish (Sprints 8-12): 25%
- Testing & refinement (Sprints 11-12): 10%
- Documentation (Sprints 13-15): 10%

### **Focus Areas:**

**Weeks 1-10:** Get everything working **Weeks 11-20:** Make it good **Weeks 21-30:** Make it great **Weeks 31-32:** Present it confidently

---

## **Budget Summary**

### **Hardware Costs:**

- ESP32: RM 32.64 ✓
- HC-SR04 Sensors (4×): RM 37.00 ✓
- Breadboard: RM 2.82 ✓
- Jumper Wires: RM 10.20 ✓
- LEDs: RM 2.40 ✓
- Foam board: RM 15
- Toy cars (4×): RM 25
- Paint/markers: RM 15
- Power bank: RM 0 (use existing)

**Total: RM 142.06** (already spent RM 85.06)

**Software Costs:**

- Firebase: Free tier (sufficient for FYP)
- Stripe: Free (test mode)
- Agora: Free tier or skip VoIP
- Google Maps API: Free (with limits)

**Total Software: RM 0 during development**

**Timeline Overview**

Month 1-2 (Weeks 1-8):

- └─ Foundation + Hardware + Authentication
- └─ Deliverable: Prototype with real-time display + login

Month 3-4 (Weeks 9-16):

- └─ Booking system + Payment
- └─ Deliverable: Complete booking flow working

Month 5 (Weeks 17-20):

- └─ QR system + Communication
- └─ Deliverable: All core features complete

Month 6-7 (Weeks 21-28):

- └─ Testing + Polish + Thesis writing
- └─ Deliverable: Polished system + draft thesis

Month 8 (Weeks 29-32):

- └─ Final thesis + Presentation prep
- └─ Deliverable: Submitted thesis + successful demo

Total: 8 months / 32 weeks

**Comparison: Waterfall vs Agile for Your Project**

| Aspect           | Waterfall (Original Plan) | Agile (This Plan)   |
|------------------|---------------------------|---------------------|
| Hardware testing | Week 5                    | Week 3-4 (Sprint 1) |

| Aspect                | Waterfall (Original Plan) | Agile (This Plan)        |
|-----------------------|---------------------------|--------------------------|
| First working demo    | Week 15+                  | Week 6 (Sprint 2)        |
| User feedback         | Week 23-24                | Week 23 + ongoing        |
| Risk discovery        | Late (testing phase)      | Early (Sprint 1-2)       |
| If hardware fails     | Major problem Week 5      | Adapt in Sprint 2        |
| Working prototype     | End of project            | Every 2 weeks            |
| Supervisor confidence | Uncertain until end       | Steady progress visible  |
| Thesis writing        | Rush at end               | Continuous documentation |

## Key Differences from Original Roadmap

### Changed:

- ❌ Real parking lot installation → ✅ Tabletop prototype
- ❌ 5 ESP32 boards → ✅ 1 ESP32 with 4 sensors
- ❌ Hardware deployment risk → ✅ Controlled demo environment
- ❌ Building permissions needed → ✅ No permissions required

### Unchanged:

- ✅ Flutter + Firebase architecture
- ✅ QR code innovation
- ✅ Booking system features
- ✅ Payment integration
- ✅ Real-time data streaming

### Better:

- ✅ Lower cost (RM 142 vs RM 400+)
- ✅ Faster to build (Week 3-4 vs Week 4-5)
- ✅ More reliable for demos
- ✅ Easier to transport
- ✅ Controlled testing environment

---



## Tips for Success

### Technical:

1. **Test incrementally** - Don't wait to test everything together
2. **Commit code daily** - Git is your friend
3. **Comment your code** - Future you will thank you
4. **Keep backups** - USB drive + cloud storage

### Academic:

1. **Document as you go** - Don't wait for thesis time
2. **Take photos/videos** - Visual evidence for thesis
3. **Track challenges** - Makes great thesis content
4. **Meet supervisor every 2 weeks** - Show sprint progress

### Practical:

1. **Manage your time** - Don't underestimate thesis writing
  2. **Start early** - Don't procrastinate
  3. **Ask for help** - Classmates, online forums, me!
  4. **Stay healthy** - Sleep, eat, exercise
- 

## Backup Plans

### If Time Runs Short:

#### Must Have (Core MVP):

-  Prototype with working sensors
-  App showing real-time parking
-  Basic booking system
-  QR code generation/scanning

#### Nice to Have (Can skip):

-  VoIP calling (use simple call button)
-  Advanced navigation (use Google Maps link)
-  Booking history (focus on active booking)
-  Reviews/ratings

## Can Mention as Future Work:

- Camera-based LPR
  - Multi-building support
  - iOS app
  - Integration with city councils
- 

## Final Checklist Before Submission

### Hardware:

- ☐ Prototype works reliably (test 20+ times)
- ☐ All components secured (nothing loose)
- ☐ Wiring is clean and professional
- ☐ LEDs all functional
- ☐ Can set up in <5 minutes

### Software:

- ☐ App runs without crashes
- ☐ All core features working
- ☐ Firebase security rules updated
- ☐ No test data in production database
- ☐ APK generated and tested

### Documentation:

- ☐ Thesis complete (all chapters)
- ☐ Code commented
- ☐ GitHub repository organized
- ☐ README with setup instructions
- ☐ Demo script prepared

### Presentation:

- ☐ PowerPoint slides ready
  - ☐ Demo rehearsed 10+ times
  - ☐ Backup video recorded
  - ☐ Q&A preparation done
  - ☐ Professional attire ready
-

## Conclusion

This Agile approach gives you:

-  **Working system every 2 weeks** (reduces stress)
-  **Early risk detection** (hardware tested immediately)
-  **Flexibility to adapt** (based on what works)
-  **Continuous progress** (supervisor sees results)
-  **Better final product**